

Digital Wallet Platform — Technical Documentation

Author: Ayetemimowa James Femi
Role: Software Engineer
Email: james.ayetemimowa@gmail.com
Phone: +234 901 763 0811
Assessment: Full-Stack Developer — Technical Assessment (Laravel + React)

Table of Contents

- 1. Executive Summary
- 2. System Overview
- 3. Architecture
- 4. Database Design
- 5. Backend — Laravel API
- 6. Frontend — React Application
- 7. Authentication & 2FA
- 8. Financial Integrity
- 9. Idempotency
- 10. Concurrency
- 11. Security
- 12. Error Handling
- 13. Testing Strategy
- 14. Deployment
- 15. API Reference
- 16. How to Use the Application
- 17. Engineering Decisions & Trade-offs
- 18. Known Limitations & Future Work

1. Executive Summary

This document describes a **Digital Wallet & Transfer Platform** built as part of a full-stack technical assessment. The platform enables users to:

- Register and authenticate with **two-factor authentication (2FA)**
- Hold balances in three currencies: **NGN, USD, USDT**
- Fund their wallet (simulated)
- Transfer funds to other users
- View transaction history and individual transaction details

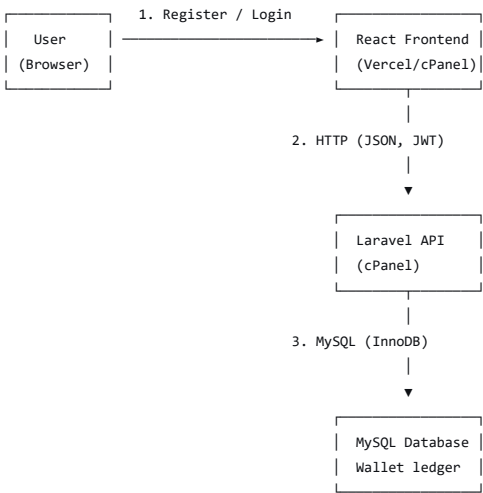
The system is split into two independently deployable components:

Component	Stack	Repository	Live URL
Backend API	Laravel 11, MySQL, JWT	DigitalWalletAPI	https://wallet.icrystal.org.ng/api
Frontend	React 19, CRA, Axios	DigitalWalletUSER	https://wallet-user.icrystal.org.ng

The implementation prioritizes **financial correctness** over feature breadth. Every money-moving operation is transactional, idempotent, and concurrency-safe. Balances are derived from an append-only ledger rather than stored as mutable columns. Failed operations leave an auditable record. Financial rules are enforced server-side without exception.

2. System Overview

2.1 What the Platform Does



2.2 Actors

- **End User** — registers, funds, transfers, views history
- **System (Backend)** — enforces limits, prevents duplicates, records every movement
- **Administrator (out-of-band)** — can flip the global kill switch by editing `financial_settings`

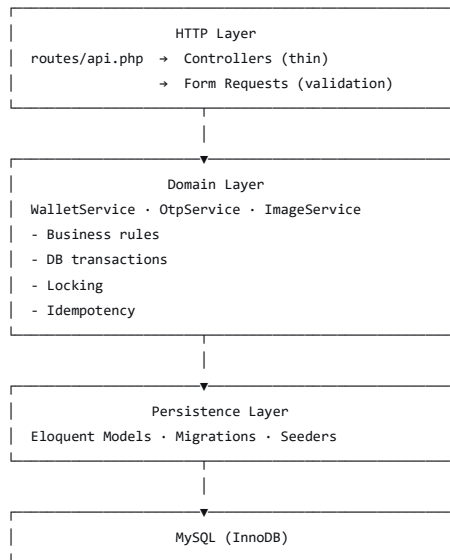
2.3 Key Capabilities

Capability	Implementation
Authentication	JWT + 6-digit email OTP (2FA)
Multi-currency wallets	NGN, USD, USDT
Funding	Simulated credit, idempotent
Transfers	P2P, atomic, locked, idempotent
History	Paginated, status-aware
Global controls	Currency enablement, kill switch, amount limits
Audit	Append-only ledger, immutable
Concurrency	MySQL GET_LOCK per user+currency

3. Architecture

3.1 Layered Backend

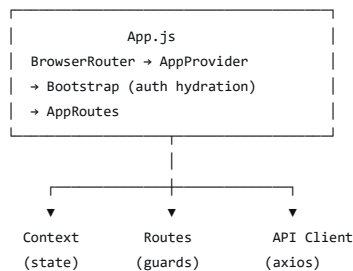
text



Why this layering matters: Controllers never touch money logic. All financial rules live in **WalletService** and are enforced regardless of which entry point calls them (HTTP, CLI, tests). This prevents the common fintech bug where a rule is enforced in one controller but bypassed in another.

3.2 Frontend Structure

text

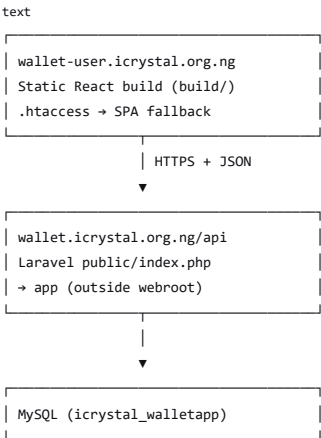


State: All authentication and user state lives in **AppContext**, accessed via `useGlobalValue()` and `useSetGlobalValue()`.

HTTP: A single Axios instance in `src/api/client.js` reads `REACT_APP_API_URL` and injects the bearer token automatically.

Routing: **AppRoutes** renders public vs. private routes based on the current user. Unauthenticated users are redirected to `/login`.

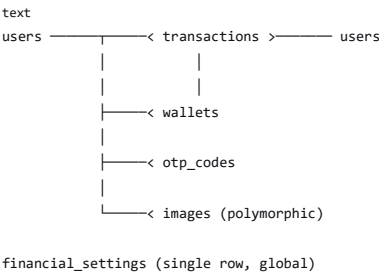
3.3 Deployment Topology



Both subdomains run on the **same cPanel server**. Laravel's project files sit **outside** the subdomain's document root; only **public/** is exposed via HTTP.

4. Database Design

4.1 Entity Relationship (conceptual)



4.2 Tables

users

Column	Type	Notes
id	BIGINT PK	
name	VARCHAR(255)	
email	VARCHAR(255) UNIQUE	
email_verified_at	TIMESTAMP NULL	
password	VARCHAR(255)	bcrypt hash
currencies	JSON	preferred currencies
timestamps		

transactions

Business event log for every transfer and funding.

Column	Type	Notes
id	BIGINT PK	
reference	CHAR(36) UNIQUE	public UUID
sender_id	FK users NULL	NULL for funding
receiver_id	FK users	
amount	DECIMAL(20,8)	
currency	ENUM(NGN,USD,USDT)	
narration	VARCHAR(255) NULL	
status	ENUM(pending,completed,failed)	
failure_reason	VARCHAR(255) NULL	
idempotency_key	CHAR(36) UNIQUE NULL	
idempotency_payload_hash	CHAR(64) NULL	
timestamps		

Indexes:

- UNIQUE(reference)
- UNIQUE(idempotency_key)
- INDEX(sender_id, receiver_id, currency, created_at) — duplicate detection
- INDEX(receiver_id, created_at) — recipient history
- INDEX(status)

wallets (append-only ledger)

Every movement of money produces a row here. **Immutable**.

Column	Type	Notes
id	BIGINT PK	
user_id	FK users	
transaction_id	FK transactions NULL	NULL for funding
currency	ENUM(NGN,USD,USDT)	
entry_type	ENUM(debit, credit)	
source	ENUM(funding, transfer, reversal)	
amount	DECIMAL(20,8)	always positive
balance_after	DECIMAL(20,8)	snapshot
created_at	TIMESTAMP	no updated_at

Indexes:

- INDEX(user_id, currency, created_at)

- INDEX(transaction_id)

financial_settings (single row)

Global control table.

Column	Type	Notes
id	BIGINT PK	
currencies	JSON	enabled currencies
transaction_allowed	ENUM(active, inactive)	global kill switch
financial_year	VARCHAR(10)	
month	TINYINT	
duplicate_window_seconds	INT	default 30
min_transfer_amount / max_transfer_amount	DECIMAL(20,8) NULL	
min_funding_amount / max_funding_amount	DECIMAL(20,8) NULL	
timestamps		

otp_codes

Column	Type	Notes
id	BIGINT PK	
user_id	FK users	
code_hash	CHAR(64)	SHA-256 of 6-digit code
purpose	VARCHAR(32)	login
attempts	TINYINT	max 5
expires_at	TIMESTAMP	+10 minutes
consumed_at	TIMESTAMP NULL	
timestamps		

images (polymorphic)

Column	Type	Notes
id	BIGINT PK	
imageable_type	VARCHAR(255)	e.g. App\Models\User
imageable_id	BIGINT	
path	VARCHAR(255)	storage-relative
disk	VARCHAR(50)	public
mime_type	VARCHAR(100)	
size	INT UNSIGNED	bytes
original_name	VARCHAR(255) NULL	

Column	Type	Notes
--------	------	-------

timestamps

Index: `INDEX(imageable_type, imageable_id)`

4.3 Why an Append-Only Ledger

Traditional wallets store a `balance` column and update it in place. That approach:

- Loses history
- Makes reconciliation hard
- Allows silent drift between the "balance" and the "true" balance

This system instead records every debit and credit. A user's balance is **derived** from the last `balance_after` in their ledger. Corrections are made by **reversing entries**, never edits. The `wallets` table is effectively an event log.

Consequences:

- `SUM(credits) == SUM(debits)` per transaction is provable and tested
- A user's full financial history is auditable
- A bug that "creates money" shows up as a test failure

5. Backend — Laravel API

5.1 Stack

- **PHP** 8.2+
- **Laravel** 11
- **MySQL** 8 (InnoDB)
- **JWT** via `tymon/jwt-auth`
- **Pest** 4 for testing

5.2 Directory Map

text

app/

Exceptions/

DomainException.php base class with error code + HTTP status

InsufficientBalanceException.php

CurrencyDisabledException.php

TransactionsDisabledException.php

InvalidRecipientException.php

PossibleDuplicateException.php

IdempotencyKeyReusedException.php

Http/

Controllers/Api/

AuthController.php

WalletController.php

TransactionController.php

SettingsController.php

ImageController.php

Requests/

RegisterRequest.php

LoginRequest.php

VerifyOtpRequest.php

FundWalletRequest.php

```

    TransferRequest.php
Models/
    User.php
    Wallet.php
    Transaction.php
    FinancialSetting.php
    OtpCode.php
    Image.php
Services/
    WalletService.php
    OtpService.php
    ImageService.php
bootstrap/app.php          wires routes, CORS, exception rendering
config/
    auth.php               JWT guard
    cors.php               frontend origin
    mail.php               SMTP config
routes/
    api.php                all endpoints
database/
    migrations/            all tables
    seeders/DatabaseSeeder.php demo data
tests/
    Feature/
        AuthTest.php
        TransferTest.php
        IntegrityTest.php
        ConcurrencyTest.php
    Pest.php               test helpers

```

5.3 Service Layer — WalletService

Two public methods:

```

php
public function fund(User $user, array $payload, string $idempotencyKey): Transaction
public function transfer(User $sender, array $payload, string $idempotencyKey): Transaction

```

Both:

1. Read `FinancialSetting::current()`
2. Guard the kill switch, currency, and amount limits
3. Create a **transactions** row with status **pending** **outside** the money-moving transaction
4. Enforce idempotency (unique key)
5. Detect duplicates within the configured window
6. Acquire wallet locks (`GET_LOCK`) in sorted user order
7. Check balance (transfer only)
8. Inside a DB transaction: write wallet entries + mark **completed**
9. On failure: mark **failed** with a reason — outside the DB transaction so it persists

5.4 Exceptions → Uniform JSON

Every domain error extends `DomainException`:

```

php
class DomainException extends Exception
{
    public function __construct(
        string $message,
        public readonly string $errorCode,
        public readonly int $status = 422,
        public readonly array $errors = [],
    ) { parent::__construct($message); }
}

```

`bootstrap/app.php` renders them uniformly:

```

json
{
  "message": "Insufficient balance.",
  "code": "INSUFFICIENT_BALANCE",
  "errors": null
}

```

5.5 Idempotency Implementation

`idempotency_key` is **UNIQUE** on `transactions`. On every money-moving request:

- The row is inserted with `status = pending` and the client's key
- On `UniqueConstraintViolationException`, the existing row is fetched
- If the payload hash matches → the stored transaction is returned (**200 OK** instead of **201 Created**)
- If the payload hash differs → **422 IDEMPOTENCY_KEY_REUSED**

The database constraint **is** the concurrency guard. There is no race window.

5.6 The pending State

`pending` exists only for the duration of a single request — microseconds in practice. It never appears in user-facing history. If the process crashes mid-flow, the entire transaction rolls back and no `pending` row remains.

5.7 Duplicate Detection

Before executing a transfer, the service checks for a matching transaction within `financial_settings.duplicate_window_seconds`:

```

php
Transaction::where('sender_id', $sender->id)
    ->where('receiver_id', $receiver->id)
    ->where('amount', $payload['amount'])
    ->where('currency', $payload['currency'])
    ->where('created_at', '>=', now()->subSeconds($window))
    ->exists();

```

If found and the client did not send `confirm_duplicate: true`, the request is rejected with **409 POSSIBLE_DUPLICATE**. This catches double-clicks and repeated identical submissions that arrive too close in time. It is deliberately a **soft** block — the client can confirm and retry.

5.8 Concurrency

Two simultaneous ₦80,000 transfers on a ₦100,000 balance must not both succeed. The service uses MySQL's `GET_LOCK('wallet_{userId}_{currency}', 10)` to serialize per wallet.

- Locks are acquired in sorted `user_id` order to avoid deadlocks
- Locks are released in a `finally` block
- Inside the lock, the balance is read and the ledger entries written atomically

The `GET_LOCK` approach was chosen over `SELECT ... FOR UPDATE` because the ledger is append-only — there is no single row to lock.

6. Frontend — React Application

6.1 Stack

- **React 19**
- **Create React App** (react-scripts 5)

- **React Router 7**
- **Axios** for HTTP
- **Bootstrap 5** for styling
- **Context API** for global state

6.2 Pages

Page	Route	Purpose
Register	<code>/register</code>	Create account, trigger OTP
Verify OTP	<code>/verify-otp</code>	Enter 6-digit code
Login	<code>/login</code>	Credentials, trigger OTP
Dashboard	<code>/dashboard</code>	Balances + recent activity
Fund Wallet	<code>/fund</code>	Simulated top-up
Transfer	<code>/transfer</code>	Send to another user
Transactions	<code>/transactions</code>	Paginated history
Transaction Detail	<code>/transactions/:reference</code>	Single transaction

6.3 State Management

All auth state lives in `AppContext`:

- **user** — hydrated from `/auth/me`
- **token** — mirrored to `localStorage`
- setters via `useSetGlobalValue()`

On app load, **Bootstrap** (inside `App.js`) checks `localStorage` for a token, calls `/auth/me`, and populates the user. On failure, it clears the token and redirects.

6.4 HTTP Client

A single **Axios** instance in `src/api/client.js`:

```
js
const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (r) => r,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

All endpoint modules (`auth.js`, `wallet.js`, `transactions.js`) use this client.

6.5 UX States

Every action page implements four states:

- **Loading** — spinner or skeleton
- **Success** — confirmation message, redirect if appropriate
- **Error** — inline error banner (uses API **message** and **code**)
- **Empty** — friendly placeholder for "no transactions yet"

Buttons are disabled during submission to prevent double-submit — a second layer of protection alongside backend idempotency.

6.6 Responsive Design

Bootstrap 5 grid handles all breakpoints:

- **Desktop** — two-column dashboard
- **Tablet** — stacked cards
- **Mobile** — single column, sticky header, tap targets $\geq 44\text{px}$

7. Authentication & 2FA

7.1 Why 2FA

The assessment requires a fintech-grade experience. 2FA protects against credential stuffing and password reuse — a common failure mode for wallet products.

7.2 Flow

text

```
| 1. Client → POST /auth/login (email, password) |
|   Server → validate credentials                |
|           → generate 6-digit code              |
|           → store SHA-256 hash in otp_codes    |
|           → email the plain code               |
|           → return { requires_otp: true, email } |
|
| 2. Client → POST /auth/verify-otp (email, code) |
|   Server → find latest active OTP for user+purpose |
|           → check not expired, not consumed, attempts < 5 |
|           → hash input, compare with stored hash |
|           → mark consumed_at                   |
|           → issue JWT                          |
|           → return { access_token, user }       |
```

7.3 OTP Rules

Rule	Value
Length	6 digits
Storage	SHA-256 hash (never plaintext)
Expiry	10 minutes
Max attempts	5 (then locked)

Rule	Value
On new issue	Previous unconsumed codes invalidated
On success	consumed_at set; single-use

7.4 JWT

Property	Value
Library	tymon/jwt-auth
Access token TTL	60 minutes
Refresh TTL	14 days
Blacklist	File cache (JWT_BLACKLIST_ENABLED=true)
Logout	Blacklists the current jti

On cPanel, file cache is used because no Redis is available.

8. Financial Integrity

8.1 Invariants Enforced

1. Every debit has a matching credit of the same amount in the same transaction
2. `wallets.balance_after` equals the running balance for that user+currency
3. A user can never spend more than they have
4. A failed transfer writes **zero** wallet entries
5. `transactions` rows are never deleted — corrections are reversals
6. Amounts are always positive in storage; direction is expressed via `entry_type`

8.2 How They Are Tested

- **IntegrityTest:**
 - `SUM(debits) == SUM(credits)` per transaction
 - `balance_after` matches a manually computed running balance
- **TransferTest:**
 - Insufficient balance → no wallet entries written, transaction marked **failed**
- **ConcurrencyTest:**
 - Ledger stays balanced across many transfers

8.3 Money Representation

All money fields are `DECIMAL(20,8)`. PHP's `bcmath` functions (`bcadd`, `bcsb`, `bcsub`, `bcadd`, `bcsb`, `bcsub`, `bcadd`, `bcsb`, `bcsub`) perform arithmetic.

Never a `float`. This prevents bugs like `0.1 + 0.2 === 0.30000000000000004`.

Eloquent casts amounts to `string` to prevent accidental float coercion in PHP.

8.4 Failed Operations

A failed transfer:

- Has a `transactions` row with `status = failed`
- Has a `failure_reason` string
- Has **zero** wallet entries
- Does not affect either party's balance

The status update is written **outside** the DB transaction so a rollback doesn't undo it.

9. Idempotency

9.1 Problem

Networks retry. Users double-click. Proxies resend. Without idempotency, the same logical request can execute twice, moving money twice.

9.2 Solution

The client sends an `Idempotency-Key` header (a UUID) on every money-moving request. The key is stored on `transactions.idempotency_key`, which is `UNIQUE`.

If the same key arrives again:

- Same payload → the original response is returned (**200** instead of **201**)
- Different payload → **422 IDEMPOTENCY_KEY_REUSED**

9.3 Client Behavior

The React app generates a UUID per submission attempt and reuses it across retries of that attempt. A new attempt (user clicks "Send" afresh) gets a new key.

9.4 Guarantee

The same logical request cannot move money twice. This is enforced at the database level — not in application code — so there is no race window.

10. Concurrency

10.1 The Scenario

A user has ₦100,000. Two ₦80,000 transfers are initiated at nearly the same instant.

Without locking: both may read ₦100,000, both pass the balance check, both debit ₦80,000 — the balance goes negative or the ledger becomes inconsistent.

10.2 Solution

MySQL's `GET_LOCK` serializes access to a per-user, per-currency critical section.

```
php
DB::select('SELECT GET_LOCK(?, 10)', ["wallet_{$userId}_{$currency}"]);
try {
    // balance check + wallet writes
} finally {
    DB::select('SELECT RELEASE_LOCK(?)', [$name]);
}
```

- **Sorted acquisition** — locks are acquired in ascending `user_id` order to prevent deadlocks between simultaneous transfers in opposite directions

- **Timeout** — 10 seconds; requests wait, then fail cleanly
- **Per-user-currency key** — different wallets don't contend

10.3 Tested

ConcurrencyTest simulates sequential conflicting transfers and asserts that only one succeeds when funds are insufficient, and that the ledger remains balanced across many transfers.

11. Security

11.1 Authentication

- Passwords hashed with bcrypt
- JWT signed with **JWT_SECRET**
- Logout blacklists the token
- OTP codes stored as SHA-256 hashes

11.2 Authorization

- Every wallet/transaction query is scoped to `auth()->user()`
- Users cannot view another user's transaction (returns 404, not 403 — avoids leaking existence)
- Users cannot initiate transfers from another account
- Financial limits are enforced server-side; the frontend is never trusted

11.3 Public Identifiers

Clients interact with **reference** (UUID), never internal **id**. This prevents enumeration attacks.

11.4 Error Responses

Uniform JSON structure. No stack traces in production. **APP_DEBUG=false**.

11.5 CORS

Only the frontend origin is allowed:

```
php
'allowed_origins' => [
    'http://localhost:3000',
    'https://wallet-user.icrystal.org.ng',
],
```

11.6 HTTPS

Both subdomains run with AutoSSL (Let's Encrypt) on cPanel.

12. Error Handling

12.1 Response Shape

```
json
{
```

```
"message": "Human-readable description.",
"code": "MACHINE_READABLE_CODE",
"errors": { "field": [...] }
}
```

12.2 Error Codes

Code	HTTP	Meaning
VALIDATION_ERROR	422	Malformed payload
UNAUTHENTICATED	401	Missing/bad JWT
TOKEN_EXPIRED	401	JWT expired
TOKEN_INVALID	401	JWT signature invalid
INVALID_CREDENTIALS	401	Bad email/password
OTP_NOT_FOUND	422	No active OTP
OTP_EXPIRED	422	OTP expired
OTP_INVALID	422	Wrong OTP
OTP_LOCKED	429	5 failed attempts
INSUFFICIENT_BALANCE	422	Sender balance too low
CURRENCY_DISABLED	422	Currency globally disabled
INVALID_RECIPIENT	422	Unknown/self recipient
POSSIBLE_DUPLICATE	409	Similar recent transaction
IDEMPOTENCY_KEY_REUSED	422	Key reused with different payload
MISSING_IDEMPOTENCY_KEY	400	Header absent
TRANSACTIONS_DISABLED	503	Kill switch off
NOT_FOUND	404	Resource missing
SERVER_ERROR	500	Unexpected server error

12.3 Frontend Handling

Each page maps error codes to contextual messages. Generic errors fall back to the API's **message**. Network errors surface a retry option.

13. Testing Strategy

13.1 Framework

Pest 4 on PHPUnit. Runs against a dedicated MySQL database (`walletapp_test`) via `.env.testing`.

13.2 Coverage

File Tests Focus

File	Tests	Focus
AuthTest.php	7	Registration, OTP flow, logout, auth guards, cross-user access
TransferTest.php	9	Happy path, insufficient balance, self-transfer, idempotency, duplicate detection, kill switch
IntegrityTest.php	2	Debits == credits; running balance consistency
ConcurrencyTest.php	2	Double-spend prevention; ledger balance across many transfers
Total	20	71 assertions

13.3 Run

```
bash
php artisan test
```

Expected:

```
text
Tests:  20 passed (71 assertions)
Duration: ~3s
```

13.4 Why MySQL for Tests

SQLite does not support `GET_LOCK`, so concurrency tests are silently skipped. Using MySQL makes them real. The trade-off is speed — SQLite in-memory is faster — but correctness is worth the seconds.

14. Deployment

14.1 Backend (cPanel)

Layout:

```
text
/home/icrystal/
  en/                                ← Laravel app (outside webroot)
    app/ config/ database/ routes/ ...
    public/                          ← only this is exposed
    vendor/
    .env
  public_html/
    wallet.icrystal.org.ng/          ← subdomain docroot
      index.php                      ← points to en/
      .htaccess
```

Steps:

1. Upload project to `en/` (above webroot)
2. Create subdomain; point docroot at a folder that has Laravel's `public/` contents
3. Copy `public/*` into the subdomain docroot; edit `index.php` to require `../en/vendor/autoload.php` and `../en/bootstrap/app.php`
4. Create MySQL database + user in cPanel
5. Upload `.env` with production values
6. Run `composer install --no-dev --optimize-autoloader` (Terminal or upload `vendor/`)
7. `php artisan key:generate` (if not already set), `php artisan jwt:secret`, `php artisan migrate --seed`
8. `chmod -R 775 storage bootstrap/cache`
9. Configure mail (cPanel SMTP)
10. Enable AutoSSL

14.2 Frontend (cPanel)

1. `npm run build` locally with `.env.production` pointing at the live API
2. Create subdomain `wallet-user.icrystal.org.ng`
3. Upload `build/*` contents to the subdomain docroot
4. Add `.htaccess` for SPA fallback
5. Enable AutoSSL

14.3 Backend CORS Update

After frontend is live, add its origin to `config/cors.php` and clear config cache.

15. API Reference

Base URL: `https://wallet.icrystal.org.ng/api`

Auth

Method	Endpoint	Auth Notes	
POST	<code>/auth/register</code>	No	Creates user, sends OTP
POST	<code>/auth/login</code>	No	Validates credentials, sends OTP
POST	<code>/auth/verify-otp</code>	No	Returns JWT
GET	<code>/auth/me</code>	Yes	Current user
POST	<code>/auth/logout</code>	Yes	Blacklists token
POST	<code>/auth/refresh</code>	Yes	New access token

Wallet

Method	Endpoint	Auth Notes	
GET	<code>/wallets</code>	Yes	Balances per currency
POST	<code>/wallets/fund</code>	Yes	Idempotency-Key required
POST	<code>/me/avatar</code>	Yes	Upload avatar
GET	<code>/me/avatar</code>	Yes	Get avatar URL
DELETE	<code>/images/{id}</code>	Yes	Delete own image

Transactions

Method	Endpoint	Auth Notes	
POST	<code>/transactions</code>	Yes	Idempotency-Key required
GET	<code>/transactions</code>	Yes	Paginated history
GET	<code>/transactions/{reference}</code>	Yes	Details (participants only)

Public

Method	Endpoint	Notes
GET	/settings/status	{ transaction_allowed, currencies }

Sample: Transfer

Request

```
http
POST /api/transactions
Authorization: Bearer <jwt>
Idempotency-Key: 9f8e7d6c-4b3a-2f1e-0d9c-8b7a6f5e4d3c
Content-Type: application/json
```

```
{
  "recipient_email": "bob@example.com",
  "currency": "NGN",
  "amount": "5000.00",
  "narration": "Lunch",
  "confirm_duplicate": false
}
```

Response 201

```
json
{
  "reference": "3c8e1b2a-...",
  "status": "completed",
  "amount": "5000.00000000",
  "currency": "NGN",
  "sender": { "id": 1, "name": "Alice" },
  "receiver": { "id": 2, "name": "Bob" },
  "narration": "Lunch",
  "created_at": "2026-09-29T10:15:00+00:00"
}
```

Idempotent replay: identical body, 200 OK.

16. How to Use the Application

16.1 Live URLs

- **Frontend:** <https://wallet-user.icrystal.org.ng>
- **API base:** <https://wallet.icrystal.org.ng/api>
- **API status:** <https://wallet.icrystal.org.ng/api/settings/status>

16.2 Registration (Recommended for Reviewers)

Because 2FA sends a 6-digit code to the email address, reviewers should **register with an email they can access**.

Steps:

1. Open <https://wallet-user.icrystal.org.ng>
2. Click **Register**
3. Enter your name, an email you can access, and a password (min 8 characters)
4. Submit
5. Check your inbox for the OTP email (subject: "Your Wallet OTP Code")
6. Enter the 6-digit code on the verification screen
7. You are logged in with a JWT — you now see your dashboard

16.3 Fund Your Wallet

1. On the dashboard, click **Fund Wallet**
2. Choose currency (NGN, USD, or USDT)
3. Enter an amount
4. Submit — the balance updates immediately, and a **funding** transaction appears in your history

16.4 Send Money

1. Click **Transfer**
2. Enter the recipient's email (they must have registered)
3. Choose currency and amount
4. Add a narration (optional)
5. Submit
6. The recipient's balance updates immediately; a **transfer** transaction appears for both parties

16.5 View History & Details

1. Click **Transactions**
2. See all transfers (as sender or recipient) with status badges
3. Click any row to see details: reference, amount, narration, status, counterparty, timestamp

16.6 Demo Accounts

If you cannot access email for the OTP, you can use the seeded demo accounts:

Email	Password	Wallet
alice@example.com	password	Pre-funded ₦100,000
bob@example.com	password	₦0

However, these accounts also require OTP. **Registering with your own email is the most reliable way to test.**

16.7 Testing Idempotency

1. Open a transfer form
2. Enter recipient, currency, amount
3. Open DevTools → Network tab
4. Click Send twice rapidly
5. Only **one** transaction appears in history — the second request returns the same result

16.8 Testing Duplicate Detection

1. Send ₦1,000 to Bob
2. Immediately attempt another ₦1,000 to Bob (within 30 seconds)
3. The second attempt returns **409 POSSIBLE_DUPLICATE**
4. Confirming sends it through

16.9 Testing the Kill Switch (Admin)

If you have access to the backend database:

```
sql
UPDATE financial_settings SET transaction_allowed = 'inactive';
```

Now every transfer and funding request returns **503 TRANSACTIONS_DISABLED**. Revert with **'active'**.

17. Engineering Decisions & Trade-offs

17.1 Append-only ledger over a mutable balance column

Chosen: ledger with derived balances

Why: auditability, provable correctness, natural double-entry

Trade-off: balance reads are slightly slower; accepted because reads are cheap and correctness is not.

17.2 DECIMAL(20,8) + bcmath over floats

Chosen: fixed-point decimals with string arithmetic

Why: floating point silently corrupts money

Trade-off: slightly more verbose PHP code (bcadd etc.)

17.3 Idempotency via unique constraint

Chosen: UNIQUE(idempotency_key) on transactions

Why: enforced at the DB — no race window

Trade-off: requires clients to send the key; documented and required on money-moving endpoints

17.4 GET_LOCK over FOR UPDATE

Chosen: GET_LOCK per user+currency

Why: the ledger is append-only; there's no single wallet row to lock

Trade-off: locks are connection-scoped; on connection pooling, must be revisited

17.5 Soft duplicate detection

Chosen: 409 with confirmation flag, not a hard block

Why: users legitimately send the same amount to the same person twice

Trade-off: requires a two-step UI for the ambiguous case

17.6 pending state hidden from users

Chosen: pending is transient, invisible

Why: money hasn't moved yet; showing it would mislead

Trade-off: none in the current synchronous scope; would matter more with async settlement

17.7 JWT over session cookies

Chosen: bearer tokens

Why: decoupled frontend and backend; works across origins

Trade-off: tokens in localStorage are XSS-sensitive; HTTP-only cookies with CSRF would be more robust in production

17.8 Email OTP over SMS

Chosen: email

Why: no third-party dependency needed; works on cPanel

Trade-off: slightly slower than SMS in practice; SMS integration is straightforward with Termii/Twilio

17.9 MySQL for tests

Chosen: MySQL test database (not SQLite)

Why: concurrency tests require GET_LOCK, unsupported by SQLite

Trade-off: slower test runs; worth it for correctness coverage

17.10 Create React App over Vite

Chosen: CRA

Why: consistency with the assessment's original stack; scaffolded by default

Trade-off: CRA is deprecated upstream; Vite would be faster and leaner

17.11 Single-row `financial_settings`

Chosen: one row, global

Why: simple, auditable, easily extended with per-user overrides later

Trade-off: no per-user customization in the current scope

18. Known Limitations & Future Work

18.1 Current Limitations

- **Funding is simulated.** No payment processor integration.
- **No FX conversion.** Sender and recipient currency must match.
- **Synchronous transfers only.** No queue or async settlement.
- **No admin API.** `financial_settings` is changed via DB or Tinker.
- **No frontend automated tests.** Manual QA only.
- **No rate limiting** on auth endpoints.
- **Token in localStorage.** XSS-sensitive.
- **No audit log of admin actions.** Kill-switch flips aren't recorded.

18.2 Near-Term Improvements

1. Add a `RateLimiter` on `/auth/login` and `/auth/verify-otp`
2. Add an admin endpoint to flip `transaction_allowed` with an audit trail
3. Add Pest tests for the frontend's critical flows (via Playwright)
4. Migrate to Vite for faster builds
5. Add SMTP rate-limit handling and retry logic

18.3 Long-Term Vision

- **Payment processor integration** — Paystack (NGN), Flutterwave (multi-currency), or a stablecoin on-ramp for USDT
 - **Real-time notifications** — WebSockets or Server-Sent Events for balance updates
 - **Per-user limits** — extend `financial_settings` with a per-user overrides table
 - **Async settlement** — queue-based processing with webhooks for final status
 - **KYC/AML** — identity verification, transaction monitoring
 - **Multi-signature approvals** — for high-value transfers
 - **Ledger reconciliation job** — periodic check that `SUM(ledger) == expected` and alerts on drift
-

Appendix A — Contact

Ayetemimowa James Femi

Software Engineer

✉ james.ayetemimowa@gmail.com

☎ +234 901 763 0811

Appendix B — Repository Links

- **Backend API:** <https://github.com/femjam/DigitalWalletAPI>
- **Frontend:** <https://github.com/femjam/DigitalWalletUSER>

Appendix C — Live URLs

- **Frontend:** <https://wallet-user.icrystal.org.ng>
- **API base:** <https://wallet.icrystal.org.ng/api>
- **API status:** <https://wallet.icrystal.org.ng/api/settings/status>